Algorithm 1: `TrainingStorage` ← `BuildNormalTargets`(`RolloutStorage`, λ)

Input: `SequentialDecisionCSR` with `trajectoryPtr`$[G+1]$, stable (m,d) rows, and one terminal-reward leaf per trajectory; scalar old outputs written by rollout

Output: one scalar `lambdaReturn`$[d]$ per decision

Allocate scalar `lambdaReturn`$_m[D_m]$ and exactly-once validity bits; **foreach** *bounded trajectory batch* I ***on device*** **do**

$g_{i,p} \leftarrow$ `LeafTable`[`leafRow`$[i]$].R_p for $(i,p) \in I \times \mathcal{P}$; Reverse segmented-scan $j \in [$`trajectoryPtr`$[i]$, `trajectoryPtr`$[i+1])$:

$$(m,d) \leftarrow \texttt{decisionAddress}[j], \quad p \leftarrow \texttt{owner}[m,d], \quad \texttt{lambdaReturn}_m[d] \leftarrow g_{i,p}, \quad g_{i,p} \leftarrow \lambda g_{i,p} + (1-\lambda)\texttt{oldV}_m[d].$$

end

/* $\widehat{V}_d$ = `lambdaReturn`$[d]$ and $\widehat{A}_d$ = `lambdaReturn`$[d]$ − `oldV`$[d]$ are views/expressions in the loss, never retained tensors. Chance transitions do not advance a player clock. */

Assert every decision was written once and the recurrence equals `core_math.tex`, Eq. (sequential-λ-return); **return** *FinalizeTrainingStorage*(*RolloutStorage*);

Algorithm 2: `TrainingStorage` ← `BuildSequentialRBTargets`(`RolloutStorage`, λ)

Input: `SequentialDecisionCSR`; RB leaves containing exact generated reaches and immutable completion inputs; old values stored on natural model rows

Definitions: Cond$(m,w) = m/w$ elementwise for $w > 0$, with exactly the registered zero-mass convention from `core_math.tex`; X_m is the exact private state width, Y_m the model-row width, and $v^{\text{old}}_{m,d}(x) =$ `oldV`$_m[d, \varphi_m(x)]$

Output: one exact `lambdaReturn`$_m[d, X_m]$ per decision and one shared completion posterior row per (trajectory, p)

Allocate `lambdaReturn`$_m[D_m, X_m]$, an append-only `SequentialPosteriorPool`, int32 `posteriorRef`$_m[D_m]$, and exactly-once validity bits; **foreach** *bounded trajectory/leaf microbatch* I ***on device*** **do**

$C_I \leftarrow$ `Complete`(`LeafTable`[`leafRow`$[I]$]); Extract only $w_{i,p} = \widehat{\mathbb{P}}(-_p \mid \mathsf{T}_i^{\mathsf{pre-c}})$ and $v_{i,p} = \widehat{V}_p(-_p, \mathsf{T}_i^{\mathsf{pre-c}})$; set $m_{i,p} \leftarrow w_{i,p} \odot v_{i,p}$; aggregate scalar diagnostics and `Release`(C_I.equity/share/stddev/valid/other outputs); Append each required $w_{i,p}$ once to `SequentialPosteriorPool`; Reverse segmented-scan the decisions of each $i \in I$:

$$(m,d) \leftarrow \texttt{decisionAddress}[j], \qquad p \leftarrow \texttt{owner}[m,d], \qquad v^{\text{old}}(x) \leftarrow \texttt{oldV}_m[d, \varphi_m(x)],$$
$$\texttt{posteriorRef}_m[d] \leftarrow \text{row}(i,p), \qquad \texttt{lambdaReturn}_m[d] \leftarrow \text{Cond}(m_{i,p}, w_{i,p}),$$
$$m_{i,p} \leftarrow \lambda m_{i,p} + (1-\lambda) w_{i,p} \odot v^{\text{old}}.$$

`Release`($v_{i,p}, m_{i,p}$, completion and indexed-row scratch); retain only the appended $w_{i,p}$ rows;

end

/* For every decision, the parent and unique sampled-child posterior are the same referenced row. The loss treats lambdaReturn as its value target and derives advantage by subtracting oldV$_m[d, \varphi_m(x)]$. V1 exposes no separate Q target or loss. */

Assert exact equality to Eq. (Rao–Blackwell-λ-return) and the sequential RB surrogate in `core_math.tex`; no target/private-state aggregation is permitted; **return** *FinalizeTrainingStorage*(*RolloutStorage*);

Algorithm 3: `TrainingStorage` ← `BuildBranchedRBTargets`(`RolloutStorage`, λ)

Input: one rooted `BranchedTreeSpan`; partition tables; RB leaves

Definitions: For node s, its children are the contiguous span $[c_s, c_s + n_s)$; at a player node the same order is its action CSR span $[a_s, a_s + n_s)$. A frontier slot is $((w_{s,p}, m_{s,p})_{p \in \mathcal{P}})$

Output: exact `valueHat`$_m[D_m, X_m]$ and, per sampled player edge, `childPosterior`$_m[A_m, X_m]$, `advantageHat`$_m[A_m, X_m]$

Compute a deterministic depth-first, postorder-compatible leaf schedule. Simulate only ready counters to obtain the exact peak live-slot count $F_\star$; allocate a fixed $F_\star$ slot arena, node-to-slot map, and ready counters /* no list growth, concatenation, or all-node $[N, P, X]$ memo */

foreach *bounded next leaf microbatch* B *in that schedule* **do**

$C_B \leftarrow$ `Complete`(`LeafTable`[B]); insert $w_{t,p} = \widehat{\mathbb{P}}(-_p \mid t)$ and $m_{t,p} = w_{t,p} \odot \widehat{V}_p(-_p, t)$ into reserved slots; aggregate scalar diagnostics and release every other completion result; **while** *the ready queue is nonempty* **do**

Gather a maximum-feasible stable batch of ready parents; **foreach** *ready parent* s *with child slots* $u = 1, \ldots, n$ **do**

$S_{w,p} \leftarrow \sum_{u=1}^{n} w_{u,p}$, $S_{m,p} \leftarrow \sum_{u=1}^{n} m_{u,p}$, $w_{s,p} \leftarrow S_{w,p}/n$, and $m_{s,p} \leftarrow S_{m,p}/n$; **if** *$s$ is a player decision with address* (m,d) *and owner* p **then**

$v^{\text{old}}(x) \leftarrow$ `oldV`$_m[d, \varphi_m(x)]$; `valueHat`$_m[d] \leftarrow$ Cond$(S_{m,p}, S_{w,p})$; **for** $u = 1, \ldots, n$ **do**

$U_u \leftarrow$ Cond$(m_{u,p}, w_{u,p})$; $B_u \leftarrow v^{\text{old}}$ if $n = 1$, otherwise $B_u \leftarrow \lambda\,$Cond$(S_{m,p} - m_{u,p}, S_{w,p} - w_{u,p}) + (1-\lambda)v^{\text{old}}$; write `childPosterior`$[a_s + u - 1] \leftarrow w_{u,p}$, `advantageHat`$[a_s + u - 1] \leftarrow U_u - B_u$;

end

$m_{s,p} \leftarrow \lambda m_{s,p} + (1-\lambda) w_{s,p} \odot v^{\text{old}}$; /* only the owner moment advances its decision clock */

Release all n child slots and insert only the contracted parent slot; increment its parent's ready counter;

end

end

end

Assert the arena contains only $\varnothing$, every leaf/target was written once, and $\varnothing$ received no special multiplicity; /* The node posterior used by value/entropy/reference-KL losses is not stored: $\widehat{P}_d = n_d^{-1} \sum_{a=a_d}^{a_d+n_d-1}$ `childPosterior`$[a]$ is derived in the loss gather. */

Assert the outputs and propagated moments equal `core_math.tex`, Eqs. (branched-backward-updates) and (branched-targets) after projecting to the fixed-intensity PPO v1 value and advantage arguments; the optional Q target is deliberately deferred; **return** *FinalizeTrainingStorage*(*RolloutStorage*);

Algorithm 4: `TrainingStorage` ← `FinalizeTrainingStorage`(`RolloutStorage`)

Validate row domains, action spans, posterior references, and exactly-once writes; For each nonempty partition m, retain fp64 $\log Z_m = \text{logsumexp}_{d \in m} \log \omega_d$ and construct one alias sampler with probabilities $\exp(\log \omega_d - \log Z_m)$; use an allocation-free uniform sampler in sequential modes; Release topology nodes, leaf reaches/inputs, completion/frontier arenas, ready counters, public-state handles, child links, and all validity/scratch storage; freeze the remaining aligned partition blocks; /* Postcondition: the only iteration-owned survivor contains model inputs, natural old model rows, exact targets/posteriors, fixed sampling metadata, and no object from which a discarded reach must be recomputed. */

return *immutable TrainingStorage*;